

利用 Sarathi-Serve 驯服 LLM 推理中的吞吐量-延迟权衡

Amey Agrawal^{*2}, Nitin Kedia¹, Ashish Panwar¹, Jayashree Mohan¹, Nipun Kwatra¹,
Bhargav S. Gulavani¹, Alexey Tumanov², and Ramachandran Ramjee¹

¹Microsoft Research India

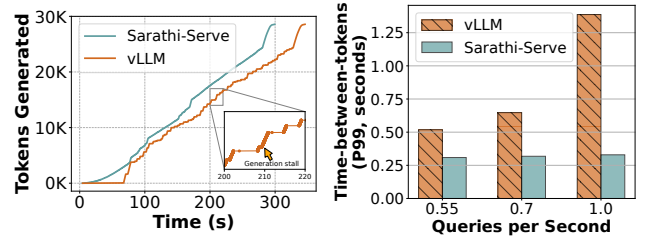
²Georgia Institute of Technology

Abstract

每个 LLM 服务请求都会经历两个阶段。第一阶段是预填充，它处理整个输入提示并生成第一个输出标记；第二阶段是解码，它逐个生成剩余的输出标记。预填充迭代延迟较高，但由于需要并行处理输入提示，因此会占用大量 GPU 计算资源。相比之下，解码迭代延迟较低，但计算资源利用率也较低，因为每次解码迭代每个请求只处理一个标记。这使得批量处理对于解码操作以及整体吞吐量都非常有效。然而，批量处理多个请求会导致预填充和解码迭代交错进行，这使得同时实现高吞吐量和低延迟变得困难。为了解决这种吞吐量-延迟权衡问题，我们引入了一个高效的 LLM 推理调度器 Sarathi-Serve。Sarathi-Serve 引入了 *chunked-prefills*，它将预填充请求分割成大小相近的块，并创建无停顿调度，从而可以在不暂停正在进行的解码操作的情况下，批量添加新的请求。无停顿调度能够利用大批量处理提升吞吐量，同时最大限度地减少批处理对延迟的影响。此外，Sarathi-Serve 中的均匀批处理改善了迭代之间的不平衡，从而最大限度地减少了流水线气泡。我们的技术在尾延迟约束下，显著提升了各种模型和硬件的推理性能。对于单 A100 GPU 上的 Mistral-7B 模型，与 vLLM 相比，我们实现了 2.6 倍的 $\times$ 服务容量提升；对于双 A100 GPU 上的 Yi-34B 模型，服务容量提升高达 3.7 倍 $\times$ 。当与 Falcon-180B 上的流水线并行处理结合使用时，Sarathi-Serve 可提供高达 5.6 倍 $\times$ 的端到端服务容量提升。Sarathi-Serve 的源代码可在 <https://github.com/microsoft/sarathi-serve> 获取。

1 简介

大型语言模型 (LLM) 在自然语言处理、问答、代码生成等众多任务中展现出了令人瞩目的能力。这促使它们在聊天机器人、搜索引擎、代码助手等众多应用中得到广泛应用。大型模型推理所需的大量 GPU 计算资源，加上其应用范围的显著扩大，使得 LLM 推理成为当今 GPU 计算的主流任务。因此，优化 LLM 推理一直是许多最新系统的研究重点。



(a) 生成停滞。

(b) 尾部延迟高。

Figure 1: Yi-34B 运行在两块 A100 GPU 上，处理来自 arxiv 摘要跟踪的 128 个请求。1a 突出显示了 vLLM 中持续数秒的众多生成停滞之一。[53]。1b 显示了负载增加对尾延迟的影响。Sarathi-Serve 在消除生成停滞的同时提高了吞吐量。

在语言学习模型 (LLM) 推理中，吞吐量和延迟的优化都是重要的目标，因为前者有助于控制服务成本，而后者则是满足应用需求的必要条件。本文指出，当前的 LLM 服务系统必须在吞吐量和延迟之间做出权衡。具体而言，批处理可以显著提高 LLM 推理的吞吐量。然而，现有系统处理多个请求的方式会导致吞吐量或延迟的妥协。例如，Figure 1b 表明，在最先进的 LLM 服务系统 vLLM [53] 中，增加负载会显著增加尾延迟。

每个 LLM 推理请求都会经历两个阶段——预填充阶段和解码阶段。预填充阶段对应于输入提示的处理，解码阶段对应于自回归词元生成。预填充阶段受计算资源限制，因为它并行处理输入提示的所有词元；而解码阶段受内存资源限制，因为它每次只处理一个请求的词元。因此，解码操作能从批处理中获益匪浅，因为更大的批次可以更有效地利用 GPU，而预填充操作则无法从批处理中获益。

当前的 LLM 推理调度器大致可以分为两类¹，即 *prefill-prioritizing* 和 *decode-prioritizing*，这取决于它们在批处理请求时如何调度预填充和解码阶段。在本文中，我们论证了这两种策略都存在根本性的缺陷，使其不适用于在线推理（参见 Figure 2）。

¹我们将最近的调度器 Splitwise [58] 和 DistServe [77] 归为第三类“非聚合式”，并在 §6 中讨论它们。

^{*}Part of this work was done during an internship at MSR India.

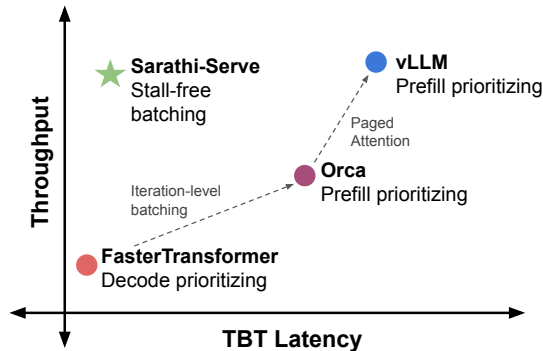


Figure 2: 当前的 LLM 服务系统需要在吞吐量和延迟之间进行权衡，具体取决于它们的调度策略。优先处理预填充可以优化吞吐量，但会牺牲 TBT（词元间隔时间）的尾部延迟，而优先处理解码则会产生相反的效果。Sarathi-Serve 通过无停顿批处理实现高吞吐量和低 TBT 延迟。（图示仅供参考，实际数值取决于模型和工作负载特性。）

传统的请求级批处理系统，例如 FasterTransformer [7]，采用 *decode-prioritizing* 调度方式。这些系统向执行引擎提交一批请求，执行引擎首先计算所有请求的预填充阶段，然后调度它们的解码阶段。只有当批处理中的所有请求都完成解码阶段后，批处理才会完成，也就是说，只要一个或多个请求正在解码，就不会调度新的预填充。这种策略优化了延迟指标“词元间隔时间”（Time-between-tokens, 简称 TBT）的推理性能——TBT 是 LLM 的一个重要性能指标。这是因为新请求不会影响正在进行解码阶段的请求的执行。然而，*decode-prioritizing* 调度器严重牺牲了吞吐量，因为即使批处理中的某些请求提前完成，执行也会以较小的批处理大小继续进行，直到最后一个请求完成。

Orca [75] 引入了迭代级批处理，请求可以以单个迭代的粒度动态地进入或退出批处理。迭代级批处理通过避免请求级批处理系统的低效性来提高吞吐量。Orca 和其他一些近期系统（例如 vLLM [23]）将迭代级批处理与 *prefill-prioritizing* 调度相结合，其中它们优先调度一个或多个请求的预填充阶段，即一旦 GPU 内存可用就立即执行。这样，*prefill-prioritizing* 调度器具有更高的吞吐量，因为优先计算预填充允许后续解码以更大的批处理大小运行。然而，优先执行预填充会导致高延迟，因为它会干扰正在进行的解码。由于预填充操作所需时间可能因提示符长度而异，因此 *prefill-prioritizing* 调度器会导致一种不良现象，我们在本文中将其称为生成停滞。例如，Figure 1a 显示，vLLM 中的生成停滞可能持续数秒以上。

传统迭代级调度系统（例如 Orca）带来的另一个挑战是流水线停滞或“气泡”现象。这些现象出现在流水线并行 (PP) 部署中，而 PP 部署对于跨多个节点扩展 LLM 推理至关重要。在具有高带宽连接的服务器（例如 NVIDIA DGX A100）上，张量并行 (TP) 可以将 LLM 部署在多达 8 个 GPU 上，从而支持大批量处理和低延迟。

然而，当超集群不可用时，TP 的延迟可能会高得令人难以接受。因此，作为 TP 的替代方案，流水线并行 (PP) 通常用于通用网络。现有系统依靠微批处理来缓解流水线停滞或“气泡”现象。然而，由于 LLM 推理的独特特性，标准的基于微批处理的调度仍然可能导致流水线气泡。具体来说，LLM 推理由长度不一的预填充和解码操作组成。因此，最终的调度结果可能在不同微批处理之间导致运行时间差异巨大，从而浪费 GPU 周期并降低系统整体吞吐量。

为了应对这些挑战，我们提出了 Sarathi-Serve，一个用于平衡可扩展在线 LLM 推理服务的吞吐量-延迟权衡的调度器。Sarathi-Serve 基于两个关键思想：*chunked-prefills* 和无停顿调度。*Chunked-prefills* 将预填充请求分割成计算量相等的块，并在多次迭代中计算提示符的预填充阶段（每次迭代使用提示符子集）。无停顿调度允许新请求加入正在运行的批处理，而不会暂停正在进行的解码操作。这涉及构建一个批次，方法是将所有正在进行的解码操作与来自新请求的一个或多个预填充块合并，从而使每个批次达到预配置的块大小。Sarathi-Serve 基于迭代级批处理，但有一个重要的区别：它会限制每次迭代中的预填充标记数量，同时允许在运行中的批次中添加新请求。这不仅限制了每次迭代的延迟，而且使其几乎与输入提示的总长度无关。这样，Sarathi-Serve 最大限度地减少了计算新预填充对正在进行的解码操作的影响，从而实现了高吞吐量和低 TBT 延迟。

此外，由 Sarathi-Serve 构建的混合批次（由预填充和解码标记组成）具有近乎均匀的计算需求。借助流水线并行性，这使我们能够创建均衡的基于微批处理的调度，从而显著减少流水线气泡并提高 GPU 利用率，进而实现高效且可扩展的部署。

我们在不同的模型和硬件上评估了 Sarathi-Serve 的性能——Mistral-7B 在单个 A100 GPU 上运行，Yi-34B 在两块 A100 GPU 上运行，采用双路张量并行；LLaMA2-70B 在八块 A40 GPU 上运行；Falcon-180B 在八块 A100 GPU 上运行，采用双路流水线和四路张量并行，并通过普通以太网连接。对于 Yi-34B，在不同的 SLO 目标下，Sarathi-Serve 的系统服务容量比 $\times$ 最多提升了 3.7 倍。同样，对于 Mistral-7B，我们实现了比 $\times$ 最多提升 2.6 倍的服务容量。Sarathi-Serve 还能减少流水线气泡，使采用流水线并行部署的 Falcon-180B 端到端服务容量提升高达 5.6 倍 ($\times$)。

本文的主要贡献包括：

1. 我们指出了当前 LLM 服务系统中的一些缺陷，尤其是在权衡吞吐量和延迟方面。
2. 我们引入了两种简单而有效的技术 *chunked-prefills* 和 *stall-free batching*，以提高 LLM 服务系统的性能。
3. 我们通过对多种模型、硬件和并行策略的广泛评估，证明了 Sarathi-Serve 的通用性，并证明它可以将模型服务容量提升一个数量级。

2 背景

本节将介绍典型的语言学习模型 (LLM) 架构及其自回归推理过程。此外，我们还将概述调度策略和重要的性能指标。

2.1 Transformer 架构

流行的大型语言模型，例如 GPT-3 [18]、LLaMA [66]、Yi [24] 等，都是仅包含解码器的 Transformer 模型，用于训练下一个词元预测任务。这些模型由一系列结构相同的层堆叠而成。每一层都包含两个模块：自注意力机制和前馈网络 (FFN)。

自注意力模块是 Transformer 架构的核心，它使序列的每个部分都能考虑之前的所有部分，从而生成上下文表示。在自注意力计算过程中，首先通过线性变换获得每个输入词元对应的查询向量 (Q)、键向量 (K) 和值向量 (V)。接下来，XMATHX 算子计算序列中所有词元之间的语义关系。这包括计算每个 Q 向量与序列中所有先前词元的 K 向量的点积，然后进行 softmax 操作以获得权重向量，最后使用该权重向量计算 V 向量的加权平均值。这种注意力计算可以在多个注意力头上执行，它们的输出通过线性变换组合。

Feed-forward network (FFN): 前馈神经网络 (FFN) 通常由两个线性变换层组成，中间穿插一个非线性激活函数。第一个线性层将维度为 h 的输入词嵌入变换到更高维度 h_2 。之后是一个激活函数，通常是 ReLU 或 GELU ([27, 46])。最后，第二个线性层将词嵌入变换回原始维度 h 。

2.2 LLM 推理过程

Autoregressive decoding: LLM 推理过程包含两个不同的阶段——*prefill* 阶段和 *decode* 阶段。预填充阶段处理用户的输入提示并生成第一个输出词。随后，解码阶段逐个生成输出词，其中上一步生成的词被传递给模型以生成下一个词，直到生成一个特殊的序列结束词。请注意，解码阶段需要访问所有先前处理过的词元关联的所有键值对才能执行注意力操作。为了避免重复计算，现代 LLM 推理系统将激活值存储在 KV 缓存中。

一个典型的 LLM 提示包含数百到数千个输入词元。在预填充阶段，所有这些提示词元在一次迭代中并行处理。并行处理可以有效地利用 GPU 计算能力。相反，解码阶段涉及模型对前一次迭代中生成的单个词元的完整前向传播。这导致计算利用率低，使得解码过程受内存限制。

生产环境的服务系统必须处理来自多个用户的并发请求。简单地按顺序处理请求会导致 GPU 计算能力严重不足。为了提高 GPU 利用率，LLM 服务系统利用批处理来并发处理多个请求。这对于解码阶段的处理尤为有效，因为在较小的批处理大小下，解码阶段的计算强度较低。较大的批处理大小可以将获取模型参数的成本分摊到多个请求中。

Algorithm 1 Request-level batching。仅当没有剩余的解码任务时才允许新的请求（第 3 行）。这优化了 TBT，但会在许多仅解码的迭代中浪费 GPU 计算资源（第 10 行），并且批次大小可能较小。

```
1: Initialize current batch  $B \leftarrow \emptyset$ 
2: while True do
3:   if  $B = \emptyset$  then
4:      $R_{new} \leftarrow \text{get\_next\_request}()$ 
5:     while  $\text{can\_allocate\_request}(R_{new})$  do
6:        $B \leftarrow B + R_{new}$ 
7:        $R_{new} \leftarrow \text{get\_next\_request}()$ 
8:      $\text{prefill}(B)$ 
9:   else
10:     $\text{decode}(B)$ 
11:     $B \leftarrow \text{filter\_finished\_requests}(B)$ 
```

Algorithm 2 Iteration-level batching (vLLM)。预填充操作会立即执行（第 8-9 行），这可能会导致正在进行的解码操作出现生成停滞（第 12 行）。

```
1: Initialize current batch  $B \leftarrow \emptyset$ 
2: while True do
3:    $B_{new} \leftarrow \emptyset$ 
4:    $R_{new} \leftarrow \text{get\_next\_request}()$ 
5:   while  $\text{can\_allocate\_request}(R_{new})$  do
6:      $B_{new} \leftarrow B_{new} + R_{new}$ 
7:      $R_{new} \leftarrow \text{get\_next\_request}()$ 
8:   if  $B_{new} \neq \emptyset$  then
9:      $\text{prefill}(B_{new})$ 
10:     $B \leftarrow B + B_{new}$ 
11:   else
12:     $\text{decode}(B)$ 
13:     $B \leftarrow \text{filter\_finished\_requests}(B)$ 
```

最近，一些互补技术被提出，旨在通过支持更大的批处理大小来优化吞吐量。Kwon 等人提出了 PagedAttention ([53])，它允许更多请求并发执行，从而消除键值缓存中的碎片。在 LLaMA、[66]、Falcon [31] 和 Yi [24] 等前沿 LLM 模型中使用多查询注意力机制 (MQA) [61] 和分组查询注意力机制 (GQA) [30] 也显著缓解了 LLM 推理中的内存瓶颈。例如，LLaMA2-70B 模型的键值缓存占用量比 LLaMA 小 $8 \times$ (-65B)。

2.3 多 GPU LLM 推理

随着模型规模的不断增长，将 LLM 扩展到多 GPU 甚至多节点部署变得至关重要 [22, 59] 此外，LLM 推理吞吐量，特别是解码阶段的吞吐量，受到 GPU 最大批处理大小的限制。因此，模型并行化可以提高推理效率，它通过将模型权重分片到多个 GPU 上，从而允许更大的批处理大小。先前的工作已经采用了张量并行化 (TP) [64] 和流水线并行化 (PP) [7, 72, 75] 来实现这一目标。

TP 通过将模型权重和键值缓存平均分配到各个 GPU 工作节点，将每一层分片到参与的 GPU 上。这样，TP 可以线性扩展每个 GPU 的批处理大小。然而，TP 的通信成本很高，因为每一层都需要两次 all-reduce 操作——一次用于注意力计算，另一次用于 FFN [64]。此外，由于这些通信操作位于关键路径上，TP 仅适用于 GPU 通过 NVLink 等高带宽互连连接的单节点环境。

与 TP 相比，PP 将模型逐层拆分，每个 GPU 负责一部分层。为了保持“流水线”中的所有 GPU 都处于忙碌状态，PP 采用了微批处理技术。这些微批处理数据在每次迭代中沿着流水线从一个阶段移动到下一个阶段。与 TP 相比，PP 具有更高的计算通信比，因为它只需为多个计算层发送一次激活值。此外，与 TP 中成本更高的 all-reduce 操作相比，PP 仅需要点对点通信操作。因此，在跨节点部署中，当高带宽互连不可用时，PP 比 TP 更高效。

对于 LLM 服务，有两个主要的延迟指标值得关注：TTFT（首令牌生成时间）和 TBT（令牌间隔时间）。对于给定的请求，TTFT 衡量从请求到达系统到生成第一个输出令牌的延迟。该指标反映了模型的初始响应速度。另一方面，TBT 衡量请求的连续输出令牌生成之间的间隔，并影响响应的整体流畅性。当系统负载较高时，低吞吐量会导致较大的调度延迟，从而导致 TTFT 升高。

此外，我们使用吞吐量指标“容量”（Capacity），它定义为系统在满足特定延迟目标的前提下所能承受的最大请求负载（每秒查询数）。更高的容量是可取的，因为它能降低服务成本。

调度器负责准入控制和批处理策略。为了便于阐述，我们将现有的 LLM 推理调度器大致分为两类——*prefill-prioritizing* 和 *decode-prioritizing*。

传统的推理引擎，例如 FasterTransformer ([7]) 和 Triton Inference Server ([17])，使用 *decode-prioritizing* 调度算法，并结合 request-level batching 和 i.e.,，选择一批请求并执行，直到该批请求全部完成 (Algorithm 1)。这种方法降低了调度框架的运行复杂度，但代价是资源利用效率低下。批次中的不同请求通常输入和输出令牌的数量差异很大。请求级调度器会用零填充较短的请求，使其长度与批次中最长的请求长度相匹配，这会导致计算资源浪费，并延长待处理请求的等待时间。

为了避免计算资源浪费，Orca 引入了一种细粒度的机制，允许请求在每次模型迭代后动态地进入和退出批次。这种方法可以显著提高系统吞吐量，目前已被许多 LLM 推理服务系统采用，例如 vLLM、TensorRT-LLM 和 LightLLM。

当前诸如 vLLM [23] 和 Orca [75] 等 iteration-level batching 系统使用 *prefill-prioritizing* 调度算法，该算法会在 GPU 内存可用时立即接受正在运行的批次中的新请求。优先处理预填充操作可以提高吞吐量，因为它增加了后续解码迭代的批次大小。

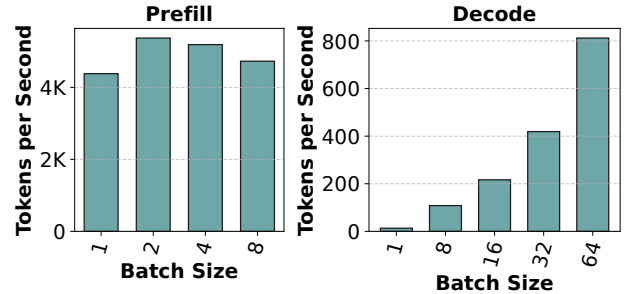


Figure 3: 在单个 A100 GPU 上运行的 Mistral-7B 中，不同批处理大小下预填充和解码阶段的吞吐量。预填充和解码实验均使用 1024 的提示长度。请注意不同的 y 轴，这表明预填充比解码效率高得多。此外，批处理几乎可以线性地提升解码吞吐量，但对预填充吞吐量的影响甚微。

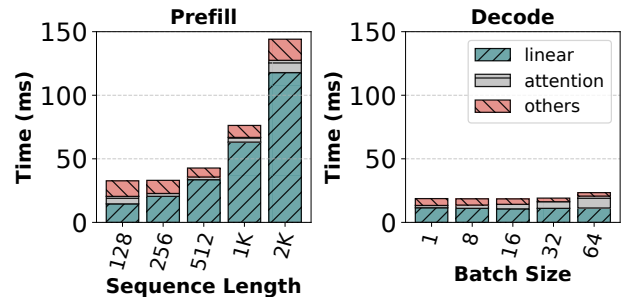


Figure 4: Mistral-7B 在单个 A100 GPU 上运行，不同输入大小下的预填充和解码时间。线性层在预填充和解码阶段都占据了大部分运行时间。由于解码批处理中的算术强度较低，1 个解码标记的线性运算成本几乎与 128 个预填充标记的成本相同。

3 动机

在本节中，我们首先分析预填充和解码操作的成本。然后，我们重点介绍吞吐量-延迟权衡以及在服务 LLM 时出现的流水线气泡。

3.1 预填充和解码的成本分析

如 §2.2 中所述，预填充阶段并行处理所有输入标记，有效地利用了 GPU 计算能力，而解码阶段一次只处理一个标记，效率非常低。Figure 3 展示了吞吐量与批处理大小的关系。我们可以观察到，解码迭代的吞吐量随批处理大小大致呈线性增长，而预填充的吞吐量即使只有一次请求也几乎达到饱和。

Takeaway-1: LLM 推理的两个阶段——预填充和解码——表现出截然不同的行为：批处理极大地提升了解码阶段的吞吐量，但对预填充阶段的吞吐量影响甚微。

Figure 4 将预填充和解码的计算时间分解为线性算子、注意力机制和其他算子，并展示了它们各自的贡献。从

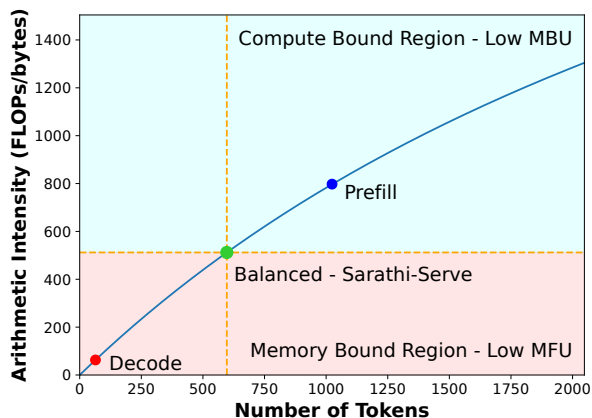


Figure 5: LLaMA2-70B 线性运算在四个 A100 GPU 上运行，不同 token 数量的算术强度趋势。解码批次的算术强度较低 (*i.e.*), 其瓶颈在于内存读取时间，导致计算利用率较低。预填充批次受计算瓶颈限制，带宽利用率也欠佳。Sarathi-Serve 通过合并解码和预填充块来形成均衡批次，从而最大化计算和带宽利用率。

图中可以看出，线性算子贡献了大部分运行时间。虽然注意力机制的开销随序列长度呈二次方增长，但即使在序列长度很高的情况下，线性算子仍然贡献了超过 80 个% 的总时间。因此，优化线性算子对于提高 LLM 推理的性能至关重要。

Low Compute Utilization during Decodes: 解码阶段的计算利用率低，浪费了 GPU 的处理能力。为了进一步理解这一点，我们分析了预填充和解码迭代的算术强度。由于 LLM 推理的大部分时间都消耗在线性算子上，因此我们将分析重点放在线性算子上。

矩阵乘法内核在进行数学运算的同时，也会进行内存访问。一次运算的总执行时间可以近似表示为 $T = \max(T_{\text{math}}, T_{\text{mem}})$ ，其中 T_{math} 和 T_{mem} 分别表示数学运算和内存读取操作所花费的时间。如果 $T_{\text{math}} < T_{\text{mem}}$ 大于 $T_{\text{math}} < T_{\text{mem}}$ ，则该运算被认为是内存密集型的。内存密集型运算的模型浮点运算利用率 (MFU) [35] 较低。另一方面，计算密集型运算的模型带宽利用率 (MBU) 较低。当 $T_{\text{math}} = T_{\text{mem}}$ 大于 $T_{\text{math}} = T_{\text{mem}}$ 时，计算带宽和内存带宽的利用率均达到最大值。算术强度量化了从内存中读取的每个字节数据所执行的数学运算次数。在最佳点，运算的算术强度与设备的浮点运算/带宽比相匹配。Figure 5 展示了在四块 A100 GPU 上运行的 LLaMA2-70B 中线性层的算术强度与批次中标记数的关系。预填充批次将线性算子权重从 HBM 内存到 GPU 缓存的获取成本分摊到大量标记上，因此具有较高的算术强度。相比之下，解码批次的计算强度非常低。Figure 6 展示了 LLaMA2-70B 中线性算子在一次迭代中的总执行时间与标记数的关系。请注意，在批次处于内存密集型状态时，执行时间在 *i.e.*，初期仅略有增加，但当批次变为计算密集型时，执行时间会线性增加 (*i.e.*)。

Takeaway-2: 解码批次在内存瓶颈下运行，导致计算利

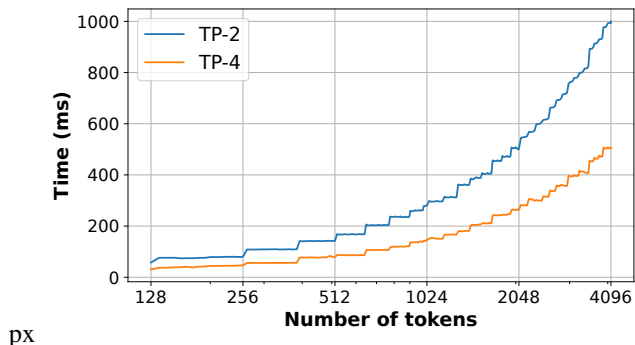


Figure 6: 在 A100(s) 上，使用不同张量并行度的 LLaMA2-70B 层，线性层执行时间与批次中标记数的关系如图所示。当标记数较少时，执行时间主要取决于从 HBM 内存中获取权重的成本。因此，在 128-512 个标记的范围内，执行时间基本保持不变，尤其是在张量并行度较高的情况下。一旦批次中的标记数超过临界阈值，操作就会受计算限制，运行时间随标记数线性增加。

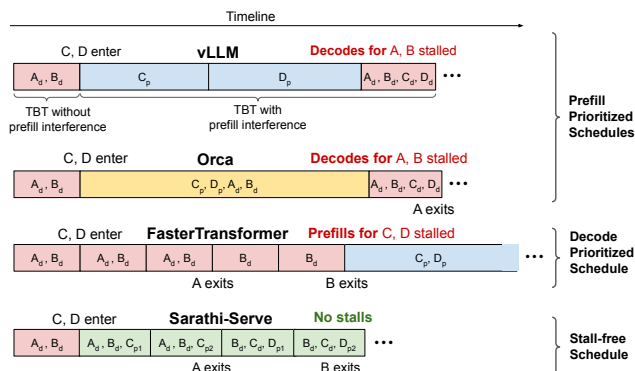


Figure 7: 当在请求的连续解码迭代之间安排一个或多个预填充操作时，就会发生生成停滞。A、B、C 和 D 代表不同的请求。下标 d 表示一次解码迭代， p 表示一次完整的预填充操作， p_0 和 p_1 表示对给定提示符的两次分块预填充操作。vLLM 通过在恢复正在进行的解码之前尽可能多地调度预填充操作来诱发生成停顿。尽管 Orca 支持混合批处理，但由于包含长提示符的批处理的执行时间仍然很高，因此无法缓解生成停顿。FasterTransformer 不会出现生成停顿，因为它会在调度新的预填充操作之前完成所有正在进行的解码操作，但由于解码批处理大小较小，吞吐量有所降低。相比之下，Sarathi-Serve 生成的调度方案既消除了生成停顿，又提供了高吞吐量。

用率不足。这意味着可以在不显著增加延迟的情况下，处理更多 token 并同时进行的解码批次处理。

3.2 吞吐量-延迟权衡

Iteration-level batching 提高了系统吞吐量，但我们发现，由于一种我们称之为生成停顿的现象，它以较高的 TBT 延迟为代价。

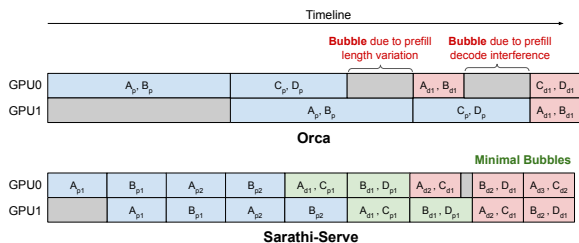


Figure 8: 在 Orca 中，对 4 个请求 (A、B、C、D) 进行双向流水线并行迭代级调度，结果显示由于批处理执行时间不均匀而导致流水线气泡的存在。Sarathi-Serve 通过创建计算时间均匀的批处理来最大限度地减少这些停顿。

Figure 7 比较了不同的调度策略。示例展示了请求 A、B、C 和 D 的时间线（从左到右）。在时间间隔开始时，请求 A 和 B 处于解码阶段，经过一次迭代后，请求 C 和 D 进入系统。Orca 和 vLLM 都使用 FCFS iteration-level batching 调度算法，并积极响应预填充请求，但它们的批次组成策略不同。Orca 支持由预填充请求和解码请求组成的混合批次，而 vLLM 仅支持包含全部预填充请求或全部解码请求的批次。尽管存在这种差异，Orca 和 vLLM 都可以通过在后续解码迭代中最大化批次大小来提高吞吐量。然而，积极调度请求 C 和 D 的预填充会延迟正在运行的请求 A 和 B 的解码，因为计算一个或多个预填充的迭代可能需要几秒钟，具体取决于输入提示的长度。因此，*prefill-prioritizing* 调度器可能会在正在进行的解码过程中引入生成停顿，从而导致高 TBT 引起的延迟峰值。

与 iteration-level batching 不同，request-level batching 系统（例如 FasterTransformer [7]）在所有已运行的请求完成解码阶段之前不会调度新的请求（Algorithm 1 中的第 3 行）。在 Figure 7 中，请求 C 和 D 的预填充操作会一直停滞，直到请求 A 和 B 都退出系统。因此，*decode-prioritizing* 系统虽然可以降低 TBT 延迟，但代价是系统吞吐量较低。例如，Kwon 等人 ([53]) 的研究表明，与 FasterTransformer 相比，采用 PagedAttention 的 iteration-level batching 可以实现高一个数量级的吞吐量。

减少 iteration-level batching 系统中延迟峰值的一种方法是使用较小的批处理大小，如 Orca [75] 中建议的那样。然而，如 §2.2 所示，降低批处理大小会对吞吐量产生不利影响。因此，现有系统不得不根据所需的服务级别目标 (SLO) 在吞吐量和延迟之间进行权衡。

Takeaway-3: 对于当前的 LLM 推理调度器而言，预填充和解码的交错执行需要在吞吐量和延迟之间进行权衡。目前最先进的系统采用预填充优先级调度，以牺牲 TBT 延迟为代价来换取高吞吐量。

3.3 流水线气泡会浪费 GPU 周期。

由于流水线并行 (PP) 相比张量并行 (TP) 具有更低的通信开销，因此它是大型模型跨节点部署的常用策略。然而，PP 的一个挑战在于，它会引入流水线气泡，也就是

GPU 闲置期，因为后续的流水线阶段必须等待前一阶段中相应微批次的完成。流水线气泡是训练任务中一个已知的问题，它出现在前向传播和反向传播之间，原因是前一阶段需要等待反向传播的结果到达。微批次处理是 PP 训练任务中常用的一种缓解流水线气泡的技术。

推理任务只需要前向计算，因此人们可能会认为微批次处理可以消除推理过程中的流水线气泡。事实上，之前关于 Transformer 推理的工作，例如 FasterTransformer 和 FastServe，都使用了微批次处理，但并没有提及流水线气泡问题。最近提出的 Orca ([75]) 也表明，迭代级调度可以消除流水线调度中的气泡（参见 [75] 中的图 8）。然而，我们的实验表明，即使采用迭代级调度，流水线气泡在 PP 算法下仍然会浪费大量的 GPU 周期 (§5.3)。

在 LLM 推理中，每个微批次（或迭代）所需的计算量（以及执行时间）可能不同，具体取决于微批次中预填充和解码标记的组成（参见 Figure 8）。我们识别出推理过程中的三种气泡：（1）类似于 PB_1 的气泡，这是由于两个连续微批次中预填充标记的数量不同而产生的；（2）类似于 PB_2 的气泡，这是由于预填充和解码阶段的计算时间不同而产生的；（3）类似于 PB_3 的气泡，这是由于微批次之间的解码计算时间不同而产生的，因为注意力成本取决于累积的上下文长度（键值缓存的大小），并且会随请求而变化。对于 Falcon-180B 处理器，执行一次包含 4k 个标记的提示需要 ≈ 1150 毫秒，而仅解码且批次大小为 32 的迭代则需要大约 ≈ 200 毫秒。这些迭代的交错进行可能会导致 ≈ 950 毫秒的延迟。这些流水线延迟会浪费 GPU 周期，直接导致服务吞吐量下降和延迟增加。由于预填充迭代的时间更长、频率更高，提示符长度和批处理大小的增加会加剧这个问题。如果我们能够确保每个微批处理执行一致的计算，就可以缓解这些流水线延迟。

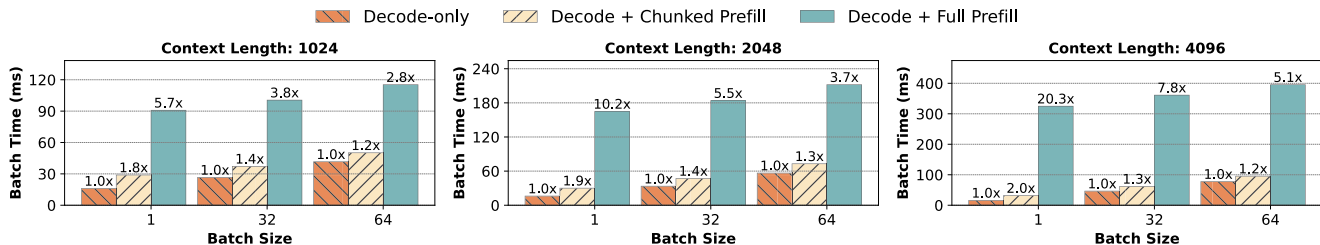
Takeaway-4: LLM 迭代的计算时间可能存在很大差异，具体取决于批处理中预填充和解码标记的组成。这在使用流水线并行时会导致显著的延迟。

4 Sarathi-Serve：设计与实现

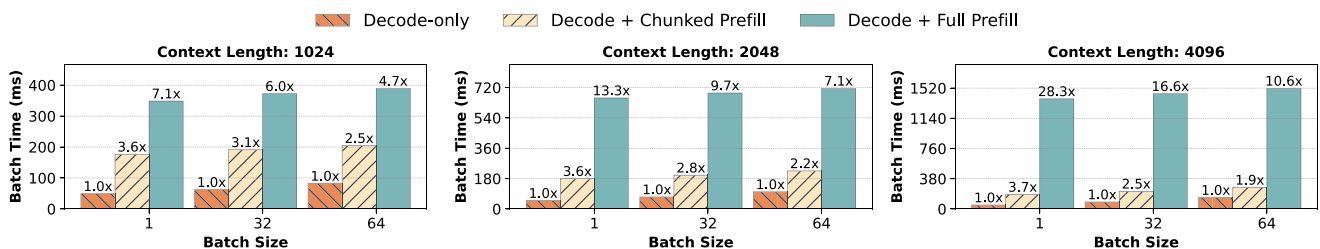
现在我们讨论 Sarathi-Serve 的设计和实现——该系统通过两种关键技术 *chunked-prefills* 和 *stall-free batching* 提供高吞吐量和可预测的尾延迟。

4.1 分块预填充

正如我们在 §3.1 中所示，解码批次严重受限于内存，但运算强度较低。运算强度的这种优势为在解码批次中叠加额外的计算提供了机会。最简单的方法是创建混合批次，将内存密集型解码与计算密集型预填充相结合。然而，在许多实际场景中，输入提示平均包含数千个词元（e.g., 和 Table 2 表明），*openchat_sharegpt4* 和 *arxiv_summarization* 数据集集中的提示大小中位数分别为 1730 和 7059。将这些较长的预填充与解码迭代相结合会导致较高的 TBT 延迟。



(a) Mistral-7B 运行在一台 A100 处理器上，令牌预算为 256。



(b) LLaMA2-70B 运行在四台 A100 处理器上，令牌预算为 512。

Figure 9: 将预填充与解码批次合并的增量成本。我们考虑两种批处理方案——(i) 解码 + 完全预填充，代表 Orca 的混合批处理，其中整个预填充与正在进行的解码一起在一次迭代中执行。(ii) 解码 + 分块预填充，代表 Sarathi-Serve，其中预填充在与正在进行的解码合并之前被分块，并且令牌预算固定。Sarathi-Serve 处理预填充令牌对解码延迟的影响要小得多。此外，Sarathi-Serve 对延迟的相对影响随着解码批次大小和上下文长度的增加而降低。

为了应对这一挑战，我们提出了一种名为 *chunked-prefills* 的技术，该技术允许在多次迭代中分小块计算较大的预填充。*Chunked-prefills* 是一种基于两个关键洞察的预填充拆分机制。首先，如 §3.1 中所述，序列长度适中的预填充请求即可有效利用 GPU 计算能力。例如，在 Figure 4 中，预填充吞吐量在序列长度约为 512 个标记时开始饱和。其次，在许多实际场景中，输入提示平均包含数千个标记 (Table 2)。这为将大型预填充请求拆分为更小的计算单元提供了机会，这些单元仍然足以利用 GPU 计算能力。在 Sarathi-Serve 中，我们利用此机制形成具有适当标记数量的批次，从而可以在解码批次中充分利用计算潜力，而不会违反 TBT SLO。

4.2 Stall-free batching

Sarathi-Serve 调度器是一个迭代级调度器，它利用 *chunked-prefills* 以及预填充和解码操作的合并来提高吞吐量，同时最大限度地减少延迟。

与 Orca 和 vLLM 不同，后者会暂停现有解码操作来执行预填充，而 Sarathi-Serve 利用解码迭代中的算术强度裕度来执行预填充，而不会延迟系统中解码请求的执行。我们将这种方法称为 *stall-free batching* (Algorithm 3)。Sarathi-Serve 首先根据用户指定的 SLO 计算一个批次中可执行的最大令牌数量预算。我们在 §4.3 中详细描述了确定此令牌预算所涉及的考虑因素。在每次调度迭代中，我们首先将下一个批次中所有正在运行的解码操作打包 (Algorithm 3 中的第 6-8 行)。之后，我们包含任何部分完成的预填充 (第 9-12 行)。只有在所有正在

运行的请求都被处理完毕后，我们才允许新的请求 (第 13-20 行)。在向批处理中添加预填充请求时，我们会计算该批处理剩余令牌预算内可容纳的最大块大小 (第 11、15 行)。通过限制每次迭代的计算负载，*stall-free batching* 确保解码操作不会因同时运行的预填充块而导致生成停滞。我们在 Figure 9 中比较了混合批处理 (有和无分块预填充) 的延迟。与仅解码的批处理相比，简单的混合批处理会导致 TBT 延迟显著增加，最高可达 28.3 (×)。相比之下，Sarathi-Serve 通过分块处理提供了更严格的延迟限制。

Figure 7 展示了 Sarathi-Serve 的调度策略的实际应用，示例与 §3.2 相同。第一次迭代仅进行解码，因为没有预填充需要计算。然而，当新的请求 C 进入系统后，Sarathi-Serve 首先将 C 的预填充分成两块，并在后续迭代中调度它们。同时，*stall-free batching* 将分块的预填充与正在进行的 A 和 B 的解码合并。这样，与现有系统不同，Sarathi-Serve 既不会阻塞解码也不会阻塞预填充，从而使 Sarathi-Serve 能够在不影响吞吐量的情况下，最大程度地避免 TBT 中的延迟峰值。此外，*stall-free batching* 与 *chunked-prefills* 结合使用，在大多数情况下还能确保均匀的混合计算批次，这有助于在使用流水线并行时减少气泡，从而实现高效且可扩展的部署。

4.3 确定令牌预算

令牌预算的确定基于两个相互竞争的因素——TBT SLO 要求和 *chunked-prefills* 开销。从最小化 TBT 的角度来看，较小的 token 预算更可取，因为预填充 token 数量越

Algorithm 3 *Stall-free batching* 与 *Sarathi-Serve* 的比较。首先, 批次被填充以进行中的解码令牌 (第 6-8 行), 并可选择性地从正在进行的解码操作中预填充一个块 (第 10-12 行)。最后, 在令牌预算范围内添加新的请求 (第 13-20 行), 以最大限度地提高吞吐量, 同时最大限度地减少延迟对 TBT 的影响。

```

1: Input:  $T_{max}$ , Application TBT SLO.
2: Initialize  $token\_budget$ ,  $\tau \leftarrow compute\_token\_budget(T_{max})$ 
3: Initialize  $batch\_num\_tokens$ ,  $n_t \leftarrow 0$ 
4: Initialize current batch  $B \leftarrow \emptyset$ 
5: while True do
6:   for  $R$  in  $B$  do
7:     if  $is\_prefill\_complete(R)$  then
8:        $n_t \leftarrow n_t + 1$ 
9:   for  $R$  in  $B$  do
10:    if  $not\ is\_prefill\_complete(R)$  then
11:       $c \leftarrow get\_next\_chunk\_size(R, \tau, n_t)$ 
12:       $n_t \leftarrow n_t + c$ 
13:    $R_{new} \leftarrow get\_next\_request()$ 
14:   while  $can\_allocate\_request(R_{new}) \wedge n_t < \tau$  do
15:      $c \leftarrow get\_next\_chunk\_size(R_{new}, \tau, n_t)$ 
16:     if  $c > 0$  then
17:        $n_t \leftarrow n_t + c$ 
18:        $B \leftarrow R_{new}$ 
19:     else
20:       break
21:
22:    $process\_hybrid\_batch(B)$ 
23:    $B \leftarrow filter\_finished\_requests(B)$ 
24:    $n_t \leftarrow 0$ 

```

少, 迭代延迟越低。然而, 较小的 token 预算会导致预填充数据过度分块, 从而造成开销, 具体表现在: 1) GPU 利用率降低; 2) 注意力操作中重复访问 KV 缓存 (我们将在下文讨论)。

在计算 *chunked-prefills* 时, 每个提示符块的注意力操作都需要访问该提示符所有先前块的 KV 缓存。即使计算成本不变, 这也会导致 GPU HBM 的内存读取次数增加。例如, 如果将预填充序列分割成 N 个块, 则第一个块的 KV 缓存会被加载 $N-1$ 次, 第二个块的 KV 缓存会被加载 $N-2$ 次, 依此类推。然而, 我们发现即使在较小的块大小下, 注意力预填充操作仍然是计算密集型操作。实际上, 由于内核启动等固定开销, 分块处理可能会带来一些额外的开销。我们对 *chunked-prefills* 在 §5.4 中的开销进行了详细研究。

因此, 在确定令牌预算时, 需要考虑预填充开销和解码延迟之间的权衡。这可以通过对不同令牌数量的批次进行一次性性能分析来解决, 并将令牌预算设置为在不违反 TBT SLO 的情况下可以打包到批次中的最大令牌数。

影响令牌预算选择的另一个因素是分块量化效应

Model	Attention Mechanism	GPU Configuration	Memory Total (per-GPU)
Mistral-7B	GQA-SW	1 A100	80GB (80GB)
Yi-34B	GQA	2 A100s (TP2)	160GB (80GB)
LLaMA2-70B	GQA	8 A40s (TP4-PP2)	384GB (48GB)
Falcon-180B	GQA	4 A100s $\times$ 2 nodes (TP4-PP2)	640GB (80GB)

Table 1: 模型和 GPU 配置 (GQA: 分组查询注意力机制, SW: 滑动窗口)。

([13])。GPU 通过将给定的矩阵分割成分块并分配给不同的线程块进行并行计算来计算矩阵乘法。这里, 每个线程块指的是一组 GPU 线程, 并计算相同数量的算术运算。因此, 当矩阵维度能被分块大小整除时, 矩阵乘法运算可以实现最大的 GPU 利用率。否则, 由于分块量化, 一些线程块会执行额外的计算 [13]。我们观察到, 在某些情况下, 分块量化会显著增加预填充的计算时间 *e.g.*, 例如, 使用 257 的块大小相比 256 的块大小, 预填充时间会增加 32 %。

最后, 在使用流水线并行时, 还应考虑令牌预算对流水线气泡的影响。较大的数据块会导致批间运行时差异增大, 从而产生流水线气泡, 最终降低系统整体吞吐量。另一方面, 选择过小的令牌预算会导致更高的开销, 因为运算强度降低, 并且存在其他固定开销。

因此, 选择合适的令牌预算是一个复杂的决策, 它取决于所需的 TBT SLO、并行配置和具体的硬件属性。我们利用 Vidur [28] (一个 LLM 推理分析器和模拟器) 来确定在特定部署场景下最大化系统容量的令牌预算。

4.4 实现

我们在开源的 vLLM [23, 53] 实现之上实现了 *Sarathi-Serve*。我们使用 FlashAttention v2 [38] 和 FlashInfer [74] 内核添加了对分页数据块预填充的支持。本文所有评估均采用 FlashAttention 后端, 因为它支持更广泛的模型集。我们还扩展了基础 vLLM 代码库, 以支持各种调度策略、分块预填充、流水线并行以及一个完善的遥测系统。我们使用 NCCL [15] 进行流水线和张量并行通信。项目源代码可在 <https://github.com/microsoft/sarathi-serve> 获取。

5 评估

我们在各种常用模型和 GPU 配置 (参见 Table 1) 以及两个数据集 (参见 Table 2) 上评估了 *Sarathi-Serve*。我们将 vLLM 和 Orca 作为基线, 因为它们代表了 LLM 推理的最新技术。我们的评估旨在回答以下问题:

1. 在特定的服务级别目标 (SLO) 约束下, 使用不同的推理服务系统 (§5.1), 一个模型副本能够承受的最大负载是多少? 该负载如何随 SLO 约束的变化而变化 (§5.2)?
2. *Sarathi-Serve* 在各种部署方式 (例如 TP 和 PP) 下的性能如何? (§5.3)

Dataset	Prompt Tokens			Output Tokens		
	Median	P90	Std.	Median	P90	Std.
<i>openchat_sharegpt4</i>	1730	5696	2088	415	834	101
<i>arxiv_summarization</i>	7059	12985	3638	208	371	265

Table 2: 用于评估的数据集。

3. *chunked-prefills* 的开销是多少? (§5.4.1)
4. 单独使用 *chunked-prefills* 和 *stall-free batching* 与同时使用它们相比, 分别会产生什么影响? (§5.4.2)

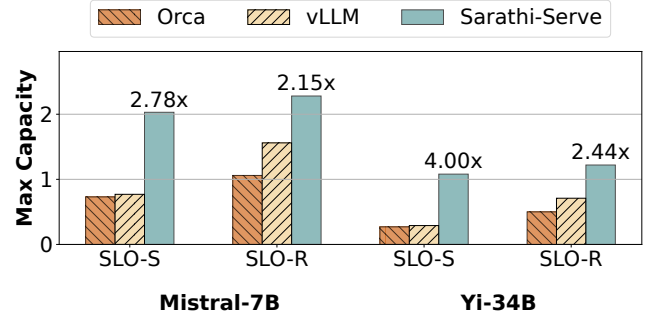
Models and Environment: 我们使用四个不同的模型 Mistral-7B、[51]、Yi-34B、[24]、LLaMA2-70B、[66]、Falcon-180B 和 [31] 来评估 Sarathi-Serve —— 这些模型在其各自的模型规模类别中名列前茅。我们使用了两种不同的服务器配置。除 LLaMA2-70B 外, 所有模型均使用 Azure NC96ads v4 虚拟机, 每个虚拟机配备 4 个 NVIDIA 80GB A100 GPU, 并通过 NVLINK 两两连接。这些虚拟机通过 100 Gbps 以太网连接。对于 LLaMA2-70B, 我们使用一台服务器, 该服务器配备 8 个两两连接的 NVIDIA 48GB A40 GPU。我们采用双向张量并行配置 (TP-2) 运行 Yi-34B, 采用混合并行配置 (TP4-PP2) 运行 LLaMA2-70B 和 Falcon-180B, 该配置包含四个张量并行工作节点和两个流水线阶段。

为了模拟真实世界的服务场景, 我们使用来自 *openchat_sharegpt4*、[68] 和 *arxiv_summarization*、[36] 数据集的请求长度特征生成跟踪数据 (Table 2)。 *openchat_sharegpt4* 跟踪数据包含用户与 ChatGPT-4 ([6]) 的对话。一次对话可能包含用户与聊天机器人之间的多轮交互。每一轮交互都作为单独的系统请求执行。这种多轮交互的特性导致提示长度的相对差异较大。相比之下, *arxiv_summarization* 是 arXiv.org 上科学出版物及其摘要 (摘要集) 的集合 ([3])。该数据集包含更长的提示信息和更低的输出词元数量方差, 代表了诸如 Microsoft M365 Copilot ([14]) 和 Google Duet AI ([10]) 等 LLM 工作负载。请求到达时间使用泊松分布生成。我们通过分别移除总长度超过 8192 和 16384 个词元的请求来过滤这些数据集中的异常值。

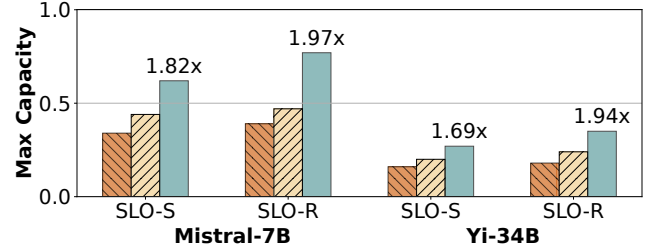
Metrics: 我们关注 TTFT 的中位数, 因为该指标每个用户请求仅获取一次; 同时关注 TBT 值的第 99 百分位数 (P99), 因为每个解码词元都会产生一个 TBT 延迟值。

Model	<i>relaxed</i> SLO	<i>strict</i> SLO
	P99 TBT (s)	P99 TBT (s)
Mistral-7B	0.5	0.1
Yi-34B	1	0.2
LLaMA2-70B	5	1
Falcon-180B	5	1

Table 3: SLOs 用于不同的模型配置。



(a) 数据集: *openchat_sharegpt4*。



(b) 数据集: *arxiv_summarization*。

Figure 10: Mistral-7B 和 Yi-34B 在严格 (SLO-S) 和宽松 (SLO-R) 延迟 SLO 下, 使用不同调度器的容量 (每秒查询数)。

5.1 容量评估

我们在两种不同的延迟配置 (*relaxed* 和 *strict*) 下, 对所有四个模型和两个数据集上的 Sarathi-Serve、Orca 和 vLLM 进行了评估。与 Patel 等人的研究类似。为了考虑模型和硬件组合固有的性能限制, 我们将 P99 TBT 上的 SLO 定义为 $5 \times$ 和 $25 \times$, 分别对应于在 *strict* 和 *relaxed* 设置下, 请求 (预填充长度为 4k, 批大小为 32) 解码迭代的执行时间, 且不进行任何预填充干扰。Table 3 显示了 SLO 绝对阈值的汇总。请注意, *strict* SLO 代表了交互式应用程序 (例如聊天机器人) 所需的延迟目标。另一方面, *relaxed* 配置示例适用于需要在可预测的时间限制内生成完整输出令牌序列, 但对单个令牌的 TBT 约束并不十分严格的系统。在所有负载实验中, 我们确保最大负载是可持续的, 即排队延迟不会过长 (我们对调度延迟中位数设置了 2 秒的限制)。

Figure 10 和 Figure 11 展示了我们容量实验的结果。在所有模型和工作负载下, Sarathi-Serve 的性能始终优于 Orca 和 vLLM。在 *strict*、SLO 和 Sarathi-Serve 的负载下, 其负载可承受比 Orca 高出 $4.0 \times$ 的水平; 在 *strict*、SLO (Yi-34B、*openchat_sharegpt4*) 的负载下, 其负载可承受比 vLLM 高出 $3.7 \times$ 的水平。对于使用流水线并行的大型模型, 由于流水线气泡较少, Sarathi-Serve 的性能分别比 Orca 和 vLLM 提升了高达 $6.3 \times$ 和 $4.3 \times$ (LLaMA2-70B, *openchat_sharegpt4*)。

我们观察到, 在大多数情况下, Orca 和 vLLM 在达到其最大可用吞吐量之前, 就会违反 P99 TBT 延迟 SLO。因此, 我们发现放宽延迟目标可以显著提高它们的模型

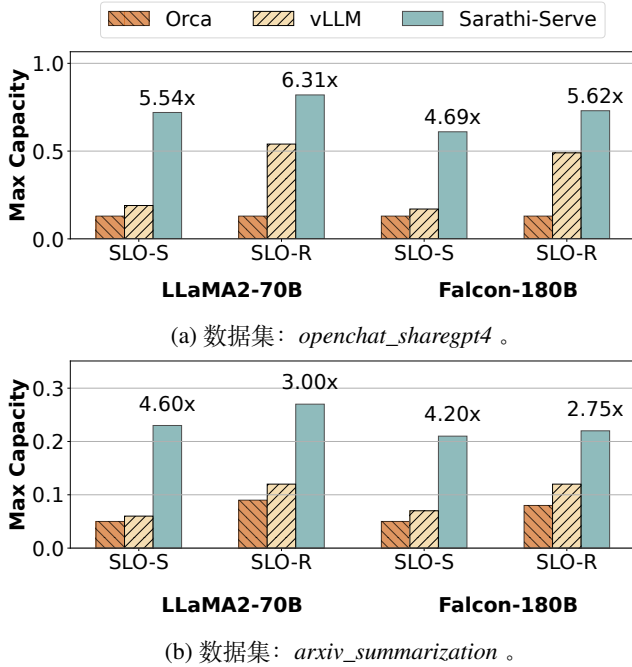


Figure 11: LLaMA2-70B 和 Falcon-180B（采用流水线并行的模型）在严格 (SLO-S) 和宽松 (SLO-R) 延迟 SLO 下，使用不同调度器的容量。

服务能力。在 Sarathi-Serve 中，可以根据所需的 SLO 调整块大小。因此，当延迟 *strict* SLO 下运行时，我们使用严格的令牌预算并将提示信息拆分成更小的块。这会略微降低系统效率，但可以实现更低的尾延迟。另一方面，当放宽延迟约束时，我们增加令牌预算以实现更高效的预填充。在 *relaxed* 和 *strict* 设置下，所有模型的 token 预算分别为 2048 和 512，但 LLaMA2-70B 和 *relaxed* 配置除外，它们使用 1536 的 token 预算以减少流水线气泡的影响。系统性能可以通过根据工作负载特征动态调整 token 预算来进一步提升。我们将此探索留待未来工作。

我们还注意到，在宽松的设置下，vLLM 的性能显著优于 Orca。原因有两方面。首先，Orca 会将多个请求一起批处理（例如，vLLM 中的最大序列长度对应最大序列长度 * 批处理大小），这在某些情况下会导致更高的尾延迟。其次，vLLM 支持比 Orca 大得多的批处理大小。Orca 的批处理大小较小是由于缺少 PagedAttention 以及处理包含大量 token 的批处理时会占用大量激活内存。

最后，请注意，与 *arxiv_summarization* 数据集相比，每个系统处理 *openchat_sharegpt4* 数据集的容量更高。这是预期的，因为 *arxiv_summarization* 数据集中的提示符要长得多——如 Table 2 所示，中位数为 7059 个标记，而 *arxiv_summarization* 数据集中位数为 1730 个标记。更长的提示符使得 Orca 和 vLLM 更容易受到延迟违规的影响，因为处理这些更长的预填充需要更长的处理时间。

5.2 吞吐量-延迟权衡

为了充分了解 LLM 服务系统中的吞吐量-延迟权衡，我们改变了 P99 TBT 延迟 SLO，并观察其对 vLLM 和 Sarathi-Serve 系统容量的影响。Figure 5.2 显示了 Mistral-7B 和 Yi-34B 模型在 *openchat_sharegpt4* 数据集上的五个不同 SLO 值的结果。

我们评估了三种不同批处理大小的 vLLM，试图解决 Yu 等人提出的 [75] 中规定的延迟-吞吐量权衡问题。由于严格的 TBT SLO 下会出现生成停滞，vLLM 的最大容量受到限制。值得注意的是，vLLM 的容量在所有三种批处理大小设置下基本保持不变。这意味着，尽管 PagedAttention 能够以高效的内存管理实现大批处理大小，但在存在延迟约束的实际场景中，由于其 *prefill-prioritizing* 调度器在延迟-吞吐量方面存在严重的权衡，vLLM 无法充分利用大批处理大小。

另一方面，Sarathi-Serve 中的延迟-吞吐量权衡可以通过调整令牌预算来精确控制。在严格的服务级别目标 (SLO) 下 (100 毫秒, Mistral-7B)，使用 512 个令牌的小预算，Sarathi-Serve 的容量比 vLLM 高 3.5 倍。对于 SLO 约束较为宽松的场景，选择更大的令牌预算 (2048) 可以使 Sarathi-Serve 更高效地运行，从而使其容量比 vLLM (1 秒, Yi-34B) 高 1.65 倍。

5.3 使流水线并行可行

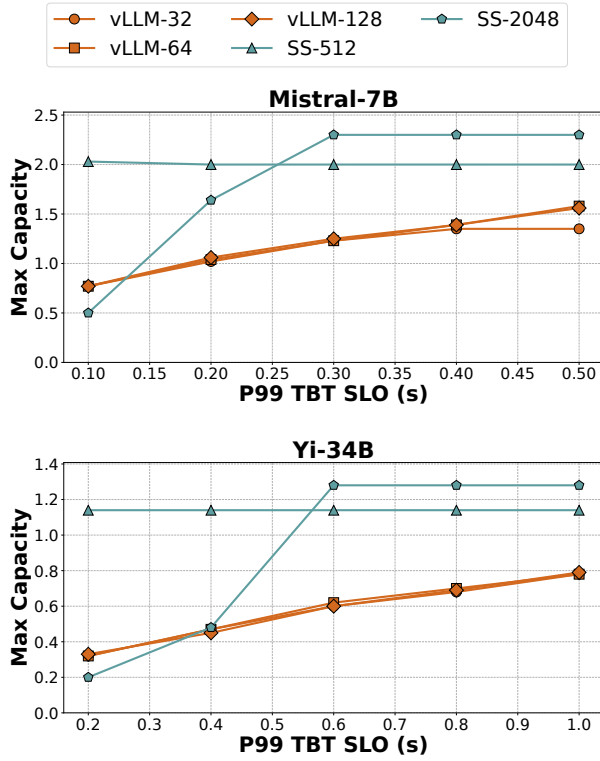
我们现在证明，Sarathi-Serve 使得在通用网络上高效地提供 LLM 推理服务成为可能，并具有高效的流水线并行性。在这些实验中，我们在两个节点上运行 Falcon-180B，每个节点配备四个 A100 GPU，并通过 100 Gbps 以太网连接。我们评估了三种配置下的模型容量：8 路 TP 的 vLLM、使用我们流水线并行实现的 vLLM 以及使用流水线并行的 Sarathi-Serve。对于 PP 配置，我们在节点内进行 4 路张量并行 (TP)，在节点间进行 2 路 PP。

Figure 13a 显示了 Falcon-180B 在纯张量并行 (TP-8) 部署下，仅解码批次的延迟，并与 TP-4 PP-2 混合同并行配置进行了比较。我们观察到，张量并行的延迟中位数高于流水线并行。这是因为跨节点全归约操作导致 TP 产生较高的通信开销。

Figure 13b 显示了 Falcon-180B 在 *openchat_sharegpt4* 数据集上的张量并行和混合同并行配置的容量。请注意，与混合同并行配置不同，即使在 *relaxed* SLO 数据集下，由于延迟较高，TP 的容量仍然很低。尽管 vLLM 在 *relaxed* 和 SLO 模式下能够支持相当高的混合同并行负载，但由于流水线气泡问题，其性能在 *strict* 模式下急剧下降。另一方面，Sarathi-Serve 利用 *chunked-prefills* 来减少微批处理之间的执行时间差异，从而避免流水线气泡问题，使得在 *relaxed* 和 SLOs 模式下容量提升了 1.48 倍 (×)，在 *strict* 和 SLOs 模式下容量提升了 3.6 倍 (×)。

5.4 消融研究

在本小节中，我们对 Sarathi-Serve 的不同方面进行了消融研究。特别是，我们感兴趣的是回答以下两个问题：



在严格的 SLO 条件下，Sarathi-Serve 的容量比 $\times$ 高出 3.5 倍（使用 *Stall-free batching*）。

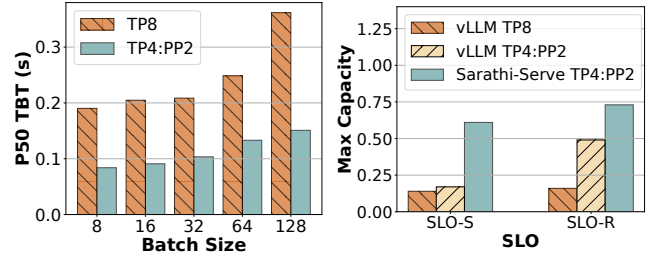
在严格的 SLO 条件下，Sarathi-Serve 的容量比 $\times$ 高出 3.5 倍（使用 *Stall-free batching*）。

Figure 12: 在 *openchat_sharegpt4* 数据集上，Mistral-7B 和 Yi-34B 模型在 vLLM 和 Sarathi-Serve 中的延迟-吞吐量权衡。我们使用三种不同的最大批处理大小（32、64 和 128）评估 vLLM。对于 Sarathi-Serve，我们考虑了 512 和 2048 的令牌预算，最大批处理大小为 128。在严格的 SLO 条件下，Sarathi-Serve 的容量比 $\times$ 高出 3.5 倍（使用 *Stall-free batching*）。

1) 分块对预填充吞吐量的影响是什么？2) 分析混合批处理和分块对延迟的影响。虽然本节仅提供部分实验的结果，但下文讨论的所有趋势在各种模型硬件组合中均保持一致。

5.4.1 *chunked-prefills* 的开销

Figure 14 显示了分块处理在 Yi-34B 中对整体预填充运行时间的开销。正如预期的那样，较小的块会带来更高的开销，这可以从 Figure 14 中逐渐降低的柱状图高度看出。然而，即使使用最小的块大小 512，我们也观察到适度的开销，最多为 $\sim 25\%$ 。而使用更大的令牌预算 2048 时，分块预填充的开销几乎可以忽略不计。



(a) TBT (Falcon-180B)。

(b) 容量 (Falcon-180B)。

Figure 13: TP 的跨节点扩展性较差。(a) 仅解码批次的中值 TBT：与节点内 4 路 TP 和跨节点 PP 相比，跨节点 TP 将中值 TBT 提高了 $2\times$ 以上。(b) 严格 (SLO-S) 和宽松 (SLO-R) 延迟 SLO 下的容量：在严格 SLO 下，与 vLLM 的仅 TP 和混合并行配置相比，Sarathi-Serve 将 Falcon-180B 的服务容量提高了 $4.3\times$ 和 $3.6\times$ 。

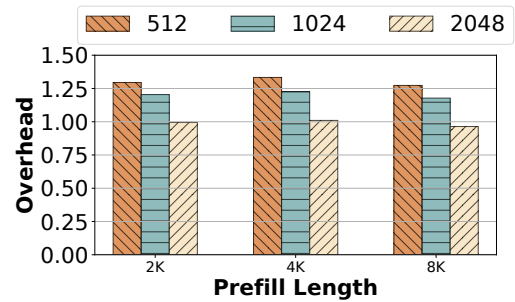


Figure 14: 针对 Yi-34B (TP-2) 的预填充计算，*chunked-prefills* 的开销已根据无分块计算的成本进行了归一化，图中展示了使用 512、1024 和 2048 三种不同分块长度的提示符长度下的结果。

5.4.2 各项技术的影响

最后，Table 4 展示了单独评估 Sarathi-Serve 的每个组件时 TTFT 和 TBT 延迟的情况。i.e., 仅使用 *chunked-prefills*，仅使用 *hybrid-batching*（混合批次，包含预填充和解码请求），以及当它们协同使用时。这些结果表明，这两种技术协同使用效果最佳：仅使用 *chunked-prefills* 会增加 TTFT，因为预填充块的效率略低；而仅使用 *hybrid-batching* 会增加 TBT，因为较长的预填充仍然会导致生成停顿。当两者结合使用时，Sarathi-Serve 在这两个维度上都提高了性能。

6 相关工作

Clipper、TensorFlow-Serving、Clockwork 和 BatchMaker 等系统研究了各种模型服务的放置、缓存和批处理策略。然而，这些系统未能解决自回归 Transformer 推理的挑战。最近，Orca、vLLM、FlexGen、FasterTransformers、LightSeq 和 TurboTransformers 等系统提出了针对 Transformer 推理的领域特定优化。FlexGen 针对资源受限的

Scheduler	openchat_sharegpt4		arxiv_summarization	
	P50 TTFT	P99 TBT	P50 TTFT	P99 TBT
hybrid-batching -only	0.53	0.68	3.78	1.38
chunked-prefills -only	1.04	0.17	5.38	0.20
Sarathi-Serve (combined)	0.76	0.14	3.90	0.17

Table 4: 针对 Yi-34B, 在两台 A100 芯片上运行, 令牌预算为 1024, 分别单独使用和同时使用 *hybrid-batching* 和 *chunked-prefills* 时, 在 128 个请求中评估了 TTFT 和 TBT 延迟 (以秒为单位)。通过同时使用 *hybrid-batching* 和 *chunked-prefills*, Sarathi-Serve 能够降低 TTFT 和 TBT。

离线场景优化 LLM 推理以提高吞吐量, 因此不适用于在线服务。FastServe [72] 提出了一种用于 LLM 推理的抢占式调度框架, 旨在最大限度地缩短作业完成时间。我们将其与 Orca 和 vLLM 进行了详细比较, 因为它们代表了 LLM 推理领域的最新技术。

最近出现的另一种方法是将预填充和解码阶段分离到不同的副本上, 如 SplitWise、DistServe 和 TetriInfer [47, 58, 77] 中所述。这些解决方案可以完全消除预填充和解码之间的干扰。然而, 分离需要在每个请求完成预填充阶段后迁移其键值缓存, 这在不同副本之间缺乏高带宽互连的情况下可能具有挑战性。此外, 这种方法也未充分利用预填充副本的 GPU 内存容量, 也就是说, 只有解码副本负责存储键值缓存。从积极的方面来看, 与速度略慢于完整预填充的分块预填充不同, 分解式方法可以以最高效率执行预填充 (因此可以获得更好的 TTFT)。我们将 Sarathi-Serve 与基于分解式的解决方案之间的定量比较留待未来工作。

最近, Sheng 等人 ([62]) 提出了对 iteration-level batching 算法的改进, 以确保多租户环境中客户端之间的公平性。FastServe ([72]) 使用基于抢占的调度机制来缓解队首阻塞。这些算法优化与我们的方法互补, 并且可以受益于 Sarathi-Serve 实现的更低的预填充解码干扰。另一个最近的系统 APIServe ([26]) 采用了 Sarathi 中的分块预填充, 以利用解码批次中浪费的计算资源进行提前预填充重新计算, 从而实现多轮 API 服务。

Improving GPU utilization for transformers: 近期研究提出了多种优化方法来提高 Transformer 模型的硬件利用率。FasterTransformer 使用了特定于模型的 GPU 内核实现。CocoNet [50] 和 [69] 旨在将计算与通信重叠以提高 GPU 利用率: 这些技术在对分布式模型使用高度张量并行时尤为有效, 因为通信时间可能远超计算时间。此外, 自注意力机制的计算成本随序列长度呈二次方增长, 因此对于长上下文而言可能非常显著。[38, 39, 60] 提出了多种技术, 通过精细的分块和工作划分来最小化自注意力机制的内存瓶颈。此外, 还探索了各种并行化策略来优化模型放置。这些技术与 Sarathi-Serve 无关。

围绕模型创新的大量工作试图解决基于 Transformer 的语言模型的不足, 或在 Transformer 的基础上实现模型架构的飞跃。例如, 多查询注意力机制 (Multi-Query Attention) 在所有注意力头之间共享相同的键值对 (KV-cache), 从而减小 KV 缓存的大小, 允许在 GPU 上处理更大的批处理大小。最近的一些研究也表明, 使用量化

(Quantization) 可以显著压缩模型大小 (Quantization)。混合专家模型 (Mixture-of-Expert Models) 的主要目标是减少每次迭代中激活的模型参数数量 (Mixture-of-Expert Models)。最近, 保留网络 (Retentive Networks) 被提出作为 Transformer 的后继者 (Retentive Networks)。与此不同, 我们专注于从 GPU 的角度解决流行的 Transformer 模型的性能问题。

7 结论

优化语言模型推理以实现高吞吐量和低延迟是理想的, 但也极具挑战性。我们对现有的 LLM 推理调度器进行了广泛的表征, 将其分为两类——*prefill-prioritizing* 和 *decode-prioritizing*。总的来说, 我们认为前一类调度器更擅长优化吞吐量, 而后一类调度器更擅长优化 TBT 延迟。然而, 当吞吐量和延迟都至关重要时, 这两类调度器都不是理想的选择。

为了解决这种权衡问题, 我们引入了 Sarathi-Serve——一个实例化了由 *chunked-prefills* 和 *stall-free batching* 组成的新方法的系统。Sarathi-Serve 将输入提示分割成更小的工作单元, 从而创建无停顿的调度。这样, Sarathi-Serve 就可以在运行的批处理中添加新的请求, 而不会暂停正在进行的解码操作。我们的评估表明, 在单个 A100 GPU 上, Sarathi-Serve 的服务能力比 Mistral-7B 提升高达 2.6 倍 (以 $\times$ 为基准); 在 8 个 A100 GPU 上, Sarathi-Serve 的服务能力比 Falcon-180B 提升高达 5.6 倍 (以 $\times$ 为基准)。

8 致谢

我们衷心感谢 OSDI 的审稿人和我们的指导老师提供的宝贵意见。本研究部分由佐治亚理工学院数据工程与科学研究所 (IDEaS) 下属的 GT Cloud Hub 提供支持, 并由微软和佐治亚理工学院新型计算层次结构研究中心 (CRNCH) 提供资金支持。

References

- [1] Amazon codewhisperer. <https://aws.amazon.com/codewhisperer/>.
- [2] Anthropic claude. <https://claude.ai>.
- [3] arxiv.org e-print archive. <https://arxiv.org/>.
- [4] Bing ai. <https://www.bing.com/chat>.
- [5] Character ai. <https://character.ai>.
- [6] Chatgpt. <https://chat.openai.com>.
- [7] Faster Transformer. <https://github.com/NVIDIA/FasterTransformer>.
- [8] Github copilot. <https://github.com/features/copilot>.

- [9] Google bard. <https://bard.google.com>.
- [10] Google duet ai. <https://workspace.google.com/solutions/ai/>.
- [11] Komo. <https://komo.ai/>.
- [12] Lightllm: A light and fast inference service for llm. <https://github.com/ModelTC/lightllm>.
- [13] Matrix multiplication background user's guide. <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [14] Microsoft copilot. <https://www.microsoft.com/en-us/microsoft-copilot>.
- [15] Nvidia collective communications library (nccl). <https://developer.nvidia.com/nccl>.
- [16] Nvidia dgx platform. <https://www.nvidia.com/en-us/data-center/dgx-platform/>.
- [17] NVIDIA Triton Dynamic Batching. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_configuration.html#dynamic-batcher.
- [18] Openai gpt-3: Understanding the architecture. <https://www.theaidream.com/post/openai-gpt-3-understanding-the-architecture>.
- [19] Perplexity ai. <https://www.perplexity.ai/>.
- [20] Replit ghostwriter. <https://replit.com/site/ghostwriter>.
- [21] Tensorrt-llm: A tensorrt toolbox for optimized large language model inference. <https://github.com/NVIDIA/TensorRT-LLM>.
- [22] Using NVIDIA's AI/ML Frameworks for Generative AI on VMware vSphere. <https://core.vmware.com/blog/using-nvidias-ai-ml-frameworks-generative-ai-vmware-vsphere>.
- [23] vllm: Easy, fast, and cheap llm serving for everyone. <https://github.com/vllm-project/vllm>.
- [24] Yi series of large language models trained from scratch by developers at 01.AI. <https://huggingface.co/01-ai/Yi-34B-200K>.
- [25] You.com. <https://you.com/>.
- [26] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Apiserve: Efficient api support for large-language model inferencing. *arXiv preprint arXiv:2402.01869*, 2024.
- [27] Abien Fred Agarap. Deep learning using rectified linear units (relu), 2019.
- [28] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for llm inference. *Proceedings of The Seventh Annual Conference on Machine Learning and Systems, 2024, Santa Clara, 2024*.
- [29] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggy-backing decodes with chunked prefills, 2023.
- [30] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023.
- [31] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models, 2023.
- [32] Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, Giri Anantharaman, Xian Li, Shuohui Chen, Halil Akin, Mandeep Baines, Louis Martin, Xing Zhou, Punit Singh Koura, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Mona Diab, Zornitsa Kozareva, and Ves Stoyanov. Efficient large scale language modeling with mixtures of experts, 2022.
- [33] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 472–487, 2022.
- [34] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [35] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker

- Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *CoRR*, abs/2204.02311, 2022.
- [36] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 615–621, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [37] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, 2017.
- [38] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- [39] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.
- [40] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.
- [41] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.
- [42] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbo Transformers: an efficient GPU serving system for transformer models. In *PPoPP '21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Virtual Event, Republic of Korea, February 27- March 3, 2021*, pages 389–402. ACM, 2021.
- [43] Elias Frantar, Saleh Ashkboos, Torsten Hoeffler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023.
- [44] Pin Gao, Lingfan Yu, Yongwei Wu, and Jinyang Li. Low latency rnn inference with cellular batching. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery.
- [45] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462, 2020.
- [46] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023.
- [47] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [48] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S. Lee, Anjali Sridhar, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. Towards moe deployment: Mitigating inefficiencies in mixture-of-expert (moe) inference, 2023.
- [49] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [50] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, page 402–416, New York, NY, USA, 2022. Association for Computing Machinery.
- [51] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego

- de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [52] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020.
- [53] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-dattention. *SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.
- [54] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 945–959, Boston, MA, July 2023. USENIX Association.
- [55] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 1–15, 2019.
- [56] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. Tensorflow-serving: Flexible, high-performance ml serving, 2017.
- [57] OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023.
- [58] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting, 2023.
- [59] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022.
- [60] Markus N. Rabe and Charles Staats. Self-attention does not need $o(n^2)$ memory, 2022.
- [61] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019.
- [62] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. *arXiv preprint arXiv:2401.00588*, 2023.
- [63] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu, 2023.
- [64] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using gpu model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [65] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023.
- [66] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [67] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [68] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. Openchat: Advancing open-source language models with mixed-quality data, 2023.
- [69] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen,

Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS 2023, page 93–106, New York, NY, USA, 2022. Association for Computing Machinery.

- [70] Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. LightSeq: A high performance inference library for transformers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers (NAACL-HLT)*, pages 113–120. Association for Computational Linguistics, June 2021.
- [71] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, 2022, 2022.
- [72] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models, 2023.
- [73] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models, 2023.
- [74] Zihao Ye, Lequn Chen, Ruihang Lai, Yilong Zhao, Size Zheng, Junru Shao, Bohan Hou, Hongyi Jin, Yifei Zuo, Liangsheng Yin, Tianqi Chen, and Luis Ceze. Accelerating self-attentions for llm serving with flashinfer, February 2024.
- [75] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.
- [76] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric. P Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. Lmsys-chat-1m: A large-scale real-world llm conversation dataset, 2023.
- [77] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Dist-

serve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.

A 工件附录

A.1

摘要 我们的开源工件可在 [和 GitHub](#) 上获取。本仓库包含我们实现的 Sarathi-Serve，以及用于运行和绘制本文所述实验的工具和脚本。

本仓库最初是 vLLM 项目的一个分支。Sarathi-Serve 是一个轻量级的高性能研究原型，其功能与开源 vLLM 并不完全相同。我们仅保留了最关键的功能，并采用了其代码库以加快研究迭代速度。

A.2

范围

本产品允许读者验证 Sarathi-Serve 论文（图表）中的结论，并提供了一种复现所述实验的方法。该产品可用于设置必要的环境、执行主要结果并进行微基准测试，从而帮助读者全面理解 Sarathi-Serve 中的关键结论。

A.3

目录

代码库结构如下：系统的主要源代码位于 /sarathi 目录下。自定义 CUDA 内核的实现位于 /csrc 目录下。所有用于复现实验的脚本位于 /osdi-experiments 目录下。最后，实验中使用的跟踪文件存储在 /data 目录下。

A.4

托管 您可以从 [GitHub](#) 获取我们的工件：

[GitHub](#)。GitHub 代码库的主分支会持续更新，但我们会在易于识别的 README 文件中提供清晰易懂的工件使用说明。所有用于复现 OSDI 论文中实验的详细说明和 README 文件都可以在 osdi-sarathi-serve 分支中找到。

A.5

要求

Sarathi-Serve 已在 A100 和 A40 GPU 上使用 CUDA 12.1 进行测试。为了便于复现，实验所使用的具体 GPU SKU 和并行策略已在与工件中图示对应的 README 文件中详细说明。